

ABHYAS

Systems & Architecture

Interview Defense Q&A Guide

Comprehensive Technical Preparation & Project Defense for 10+ LPA Roles

Project Scope Overview

This defense guide breaks down the unified codebase architecture of **ABHYAS** (Conversational Interview Terminal), **ThinkEra** (Sandboxed Compiler Playground), and **TechEra** (Automated Content Moderation Engine). It covers advanced design patterns, secure sub-process execution, database cascades, prompt pipeline engineering, and responsive browser layout debugging.

Jayesh Kumar
Full-Stack & AI Systems Engineer

May 29, 2026

Contents

1	Core Systems Design & Motivation	2
1.1	Q1: Why did you build ABHYAS? What specific problem does it solve, and how does it differ from traditional mock interview platforms?	2
1.2	Q2: Walk me through the high-level system architecture of the unified ABHYAS suite.	2
2	Backend Concurrency & API Engineering	3
2.1	Q3: Why did you choose Node.js and Express for the server backend instead of a Python-based framework like Django or FastAPI?	3
3	macOS Environment & Dependency Resolution	3
3.1	Q4: During early iterations, the python-based voice assistant module (VANI) crashed due to macOS-specific wheel compilation failures (e.g. pyobjc, pyaudio). How did you strategically redesign the architecture to solve this?	3
4	Sandboxed Compilation & Security Design	4
4.1	Q5: How did you implement secure, in-browser code evaluation for ThinkEra? What security hazards did you manage, and how did you prevent resource starvation?	4
5	Database Schema Design & Transactional Integrity	4
5.1	Q6: In TechEra/ThinkEra, you implemented an automatic content moderation and "two-strike ban" system. How did you design the database schemas and foreign key cascades to enforce data integrity?	4
6	AI Integration & Prompt Engineering	5
6.1	Q7: Walk me through the prompt engineering pipeline of ABHYAS. How does it handle multi-turn conversational history and document context injection using the Gemini SDK?	5
7	Frontend Performance & Layout Engineering	6
7.1	Q8: You solved a layout bug where the left context panel was shorter than the right terminal window. What caused this height discrepancy, and how did you resolve it mathematically?	6
7.2	Q9: What are the performance and privacy implications of using client-side document extraction (PDF.js) instead of server-side multipart uploads?	6
7.3	Q10: How does ABHYAS persist practice logs? Walk me through the file synchronization model.	7

1 Core Systems Design & Motivation

1.1 Q1: Why did you build ABHYAS? What specific problem does it solve, and how does it differ from traditional mock interview platforms?

Answer: Traditional preparation tools are either **passive** (reading LeetCode/behavioral spreadsheets) or **high-friction** (expensive, scheduled human mock interviews). This creates a massive practice vacuum where candidates struggle to convert theoretical knowledge into coherent verbal and logical pitches.

I engineered **ABHYAS** (formerly VANI Prep Hub) as an **interactive, offline-first browser terminal mock coach**. ABHYAS resolves this friction by acting as a real-time conversational technical and behavioral coach:

1. **Resume & Context Anchoring:** Candidates upload their target Job Description or Resume PDF. ABHYAS extracts the text client-side and dynamically anchors all mock questions and architectural critiques to their actual background.
2. **Unified Sandboxing:** It integrates direct conversational interview practice with secure compiler sandboxes (**ThinkEra**) so candidates can practice coding and system design in one seamless window.
3. **Low-Friction Cozy Interface:** It replaces heavy native voice synthesis engine overheads with zero-latency browser Web Speech APIs and an eye-comfortable graphite console, making continuous mock preparation highly accessible.

1.2 Q2: Walk me through the high-level system architecture of the unified ABHYAS suite.

Answer: The system is built as a highly responsive, decoupled **Client-Server Web Application** designed for 100% local deployment speed and data privacy:

1. Frontend Layer (Single Page Application):

- Styled using premium Vanilla CSS utilizing a dual-visual paradigm (Frosted cream pastel boundaries surrounding a deep, eye-safe graphite terminal ‘1a191d’).
- Integrates **PDF.js** via CDN and HTML5 **FileReader APIs** to extract document text asynchronously directly in the browser, bypassing server file storage and latency.
- Handles syntax parsing and terminal log visualization (headers, checklists, block-quotes, and dynamic readiness score badges).

2. Backend Server Layer (Node.js & Express):

- Hosts API routes for mock compiler runs (‘POST /api/run’), AI-contextual DSA hints (‘POST /api/mentor/hint’), and the conversational preparation pipeline (‘POST /api/abhyas/chat’).
- Manages standard OS child processes securely to compile and evaluate user-submitted algorithms against strict database test cases.

3. AI Orchestration Pipeline (Google Gemini SDK):

- Hooks into Google’s ‘gemini-2.5-flash’ model using the new unified ‘GoogleGenAI’ client SDK.
- Maintains multi-turn conversation memory arrays on the server, injecting system instruction matrices for Hinglish replies, STAR behavioral scores, and interactive Markdown checklists.

Unified Architectural Data Pipeline

Browser (PDF.js / input) -> Express (server.js) -> Gemini 2.5 Flash SDK -> System Prompt Execution (Hinglish/Scores) -> Client Terminal Stream Rendering

2 Backend Concurrency & API Engineering

2.1 Q3: Why did you choose Node.js and Express for the server backend instead of a Python-based framework like Django or FastAPI?

Answer: For a development prep hub that involves concurrent compilation sub-processes, real-time AI streams, and fast I/O document parsing, Node.js was selected due to its **Event-Driven, Non-Blocking Asynchronous I/O model**:

- **Concurrency Handling:** Node's single-threaded event loop utilizes the 'libuv' worker pool to manage long-running network calls (like querying the Gemini API) and sandboxed compilation sub-processes asynchronously without blocking the main execution path.
- **Resource Efficiency:** In an interview playground environment, multiple API calls are fired concurrently. Django's synchronous, thread-per-request paradigm would consume significant RAM and CPU context-switching overhead, whereas Node handles thousands of concurrent requests with a minimal memory footprint.
- **Unified Language Stack:** Writing both the client interaction logic and the server compilation routing in JavaScript/Node.js reduces cognitive overhead and allows instant data serialization (JSON) across the compiler, database schemas, and AI prompts.

3 macOS Environment & Dependency Resolution

3.1 Q4: During early iterations, the python-based voice assistant module (VANI) crashed due to macOS-specific wheel compilation failures (e.g. pyobjc, pyaudio). How did you strategically redesign the architecture to solve this?

Answer: The initial python voice engine (utilizing pyaudio, LiveKit, and macOS-native 'pyobjc-core') failed to compile during standard pip installations because macOS isolates wheel builds and lacks direct headers for C-based audio drivers in modern Python 3.12+ environments.

Instead of fighting compiler flags or forcing complex Docker audio-forwarding scripts on the user, I performed an **architectural bypass** by decoupling native python dependencies completely and moving all client-side multimedia processing to the web browser:

1. **Zero-Latency Vocal Synthesis:** Migrated the vocal coach from a Python subprocess call to the native browser **Web Speech Synthesis API** ('window.speechSynthesis'). This leveraged high-quality, pre-installed OS voices (like Apple's Samantha) with zero local package downloads and 0ms server communication latency.
2. **Client-Side Document Parsing:** Shifted file uploads from heavy server-side multi-part libraries (which require file stream writing) to the browser **HTML5 FileReader API** and a client-side **PDF.js CDN worker pipeline**.

3. **Impact:** This reduced our server boot script from an unstable, multi-gigabyte Python workspace into a clean, lightweight Node.js Express script, ensuring 100% cross-platform compatibility and absolute deployment ease.

4 Sandboxed Compilation & Security Design

4.1 Q5: How did you implement secure, in-browser code evaluation for ThinkEra? What security hazards did you manage, and how did you prevent resource starvation?

Answer: Evaluating arbitrary user code on a shared server presents massive security hazards, primarily **Remote Code Execution (RCE)** (where the user deletes files or executes system calls) and **Resource Starvation** (where infinite loops or memory leaks consume 100% server CPU/RAM).

To mitigate these risks safely in local development:

1. **Sub-Process Sandbox Isolation:** I engineered the Node.js backend to spawn isolated OS sub-processes using `'child_process.exec'` (or `'spawn'` in production containers) with resource-restricted permissions.
1. **CPU execution timeout constraints:** To handle infinite loops, I implemented a strict timer guard. If the sandboxed compiler execution exceeds **2.0 seconds**, the Node thread forcefully kills the subprocess using standard POSIX signals:

```
// Node.js process guard configuration
const execOptions = {
  timeout: 2000, // Timeout after 2s
  killSignal: 'SIGKILL', // Kill child forcefully
  maxBuffer: 1024 * 1024 // Limit standard output buffer to 1MB
};
```

2. **Output Assertions Memory Safeguard:** Standard output (`'stdout'`) is captured, sanitized of system characters, and directly asserted against official test cases fetched from our mock PostgreSQL database, returning detailed compilation JSON blocks.

5 Database Schema Design & Transactional Integrity

5.1 Q6: In TechEra/ThinkEra, you implemented an automatic content moderation and "two-strike ban" system. How did you design the database schemas and foreign key cascades to enforce data integrity?

Answer: The moderation engine must automatically warn violating accounts and completely purge banned users. If a user is banned and deleted, leaving "orphan" rows of their posts, streaks, and coding attempts will bloat database storage and cause serious foreign key (FK) constraint violations.

I designed the PostgreSQL schema to maintain absolute transactional integrity using Clerk-synchronized user webhooks and explicit **ON DELETE CASCADE** foreign keys:

```
-- PostgreSQL User Purging Cascade Schema
CREATE TABLE users (
  id VARCHAR(255) PRIMARY KEY,
  clerk_id VARCHAR(255) UNIQUE NOT NULL,
  warning_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE post_attempts (
  id SERIAL PRIMARY KEY,
  user_id VARCHAR(255) REFERENCES users(id) ON DELETE CASCADE,
  code_snippet TEXT,
  status VARCHAR(50),
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_streaks (
  user_id VARCHAR(255) PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  streak_count INT DEFAULT 0,
  last_active TIMESTAMP
);
```

Mechanics: When a user receives their second warning via a serverless Next.js API scan:

- A single ‘DELETE FROM users WHERE id = :userId’ is executed using Supabase/PostgreSQL service roles.
- Due to ‘ON DELETE CASCADE’ constraints, PostgreSQL **automatically deletes** all associated attempts, Streaks, and community posts transactionally in a single operation, preventing orphaned records and ensuring 100% schema integrity.

6 AI Integration & Prompt Engineering

6.1 Q7: Walk me through the prompt engineering pipeline of ABHYAS. How does it handle multi-turn conversational history and document context injection using the Gemini SDK?

Answer: Orchestrating a conversational technical coach requires strict output formatting, continuous dialog memory, and robust grounding.

I structured the system to execute the following pipeline inside ‘server.js’ route ‘/api/abhyas/chat’:

1. **Context Grounding:** If the candidate has uploaded a Resume PDF, the extracted text is pre-appended to the current user query:

```
[REFERENCE CONTEXT DOCUMENT PROVIDED BY USER]:
{EXTRACTED RESUME TEXT}

[USER INSTRUCTION]:
{USER QUESTION/PITCH}
```

2. **Continuous Multi-Turn Memory:** The client sends its active conversational history array. I map this history dynamically into the Gemini SDK’s standard list schema to preserve contextual continuity:

```
const contents = history.map(msg => ({
  role: msg.role === 'user' ? 'user' : 'model',
  parts: [{ text: msg.content }]
}));
```

3. **System Instruction Bounds:** I configured the 'GoogleGenAI' models to inject a strict system prompt directing the AI to speak in **Hinglish**, output a distinct rating badge **[ABHYAS SCORE: X/10]**, construct interactive study guides with ASCII flowcharts, and close with an active recall challenge.

7 Frontend Performance & Layout Engineering

7.1 Q8: You solved a layout bug where the left context panel was shorter than the right terminal window. What caused this height discrepancy, and how did you resolve it mathematically?

Answer: The height discrepancy was caused by a **nested flexbox height propagation bug** in modern web engines (Blink/WebKit).

The outer container `.workspace` used a viewport height: `height: calc(100vh - 58px)`. However, the nested `.context-panel` was styled as a column flex container but lacked an explicit height parameter (defaulting to `height: auto` or relative stretch bounds). * Because the nested column flex container did not have a defined, resolved height constraint, its child `textarea.doc-textarea` (which had `flex: 1`) **refused to stretch** because there was no computed parent bounds to fill, collapsing the card to the height of its minimum content.

I resolved this mathematically by applying explicit, viewport-relative heights to both panels inside the stylesheet:

1. **Header Constraints:** The header padding is `16px` top and bottom, and logo text height is `25px`, resulting in an exact header height of **58px**.
2. **Workspace Padding:** The workspace wrapper padding is `24px` top and bottom (total **48px**).
3. **Mathematical Bound:** Subtracting the header and padding from the total viewport height:

$$PanelHeight = 100vh - 58px - 48px = 100vh - 106px$$

4. **Implementation:** Setting `height: calc(100vh - 106px);` on both `.context-panel` and `.terminal-panel` explicitly bypassed all flex constraints, aligning the bottom borders perfectly across Chrome, Safari, and Brave, and successfully stretching the textarea to fill the workspace.

7.2 Q9: What are the performance and privacy implications of using client-side document extraction (PDF.js) instead of server-side multipart uploads?

Answer: Using client-side PDF.js parsing offers massive structural advantages:

- **Zero Server Overhead:** Standard server parsing requires multipart form uploads, writing binary files to temp folders, and loading heavy binary modules (like `pdf-parse`) in Node.js, which consumes significant RAM. Client-side extraction does all CPU-heavy parsing on the user's browser, leaving the server at 0

- **Privacy Security:** The binary PDF is never sent over the network or stored on a database. Only the raw extracted text is passed to the AI chat endpoint, preventing leakages of personal files and assuring security.
- **Reduced Latency:** Sending parsed raw text vs sending a 2MB binary PDF reduces network payloads by **90%**, cutting API transmission latency significantly.

7.3 Q10: How does ABHYAS persist practice logs? Walk me through the file synchronization model.

Answer: To track metrics without requiring a complex cloud database installation during offline prep, I engineered a **lightweight, filesystem-based logging synchronized ledger**:

- The Node server exposes two secure endpoints: 'GET /api/coach/history' and 'POST /api/coach/log'.
- When a user completes a mock draft, a POST request carries their self-rating, active question category, duration in seconds, and custom reflections.
- The backend uses Node's filesystem module ('fs') to read and write directly into a local JSON database 'coach_history.json'.
- If the database file does not exist, the backend dynamically handles parent directory generation using 'fs.mkdirSync' and initializes an empty ledger, ensuring seamless cross-terminal logs.